

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

ОТЧЁТ
по лабораторной работе № 6
Тема: «Работа с файлами, отображенными в память»

Выполнил
студент группы 450501
Томашевский Т.К.

Проверил
Поденок Л.П.

Минск 2026

1 УСЛОВИЕ ЛАБОРАТОРНОЙ РАБОТЫ

Написать многопоточную программу `sort_index` для сортировки вторичного индексного файла таблицы базы данных, работающую с файлом с использованием отображение файлов в адресное пространство процесса. Программа должна запускаться следующим образом:

```
$ sort_index memsize blocks threads filename
```

Параметры командной строки:

`memsize` – размер рабочего буфера, кратный размеру страницы (`getpagesize(2)`);

`blocks` – порядок разбиения буфера (количество блоков);

`threads` – количество потоков (от `k` до `N`), где `k` – количество процессорных ядер, `N` – максимальное количество потоков ($k \leq N \leq 8k$).

`filename` – имя файла

Количество блоков должно быть степенью двойки и превышать количество потоков не менее, чем в 4 раза (при $k = 4$ количество блоков не должно быть менее 16). Соответственно, размер файла должен удовлетворять указанным ограничениям.

Для целей тестирования следует написать программу `gen`, которая будет генерировать неотсортированный индексный файл, и программу `view` для отображения индексного файла на `stdout`.

2 ФУНКЦИОНАЛЬНАЯ СТРУКТУРА ПРОЕКТА

2.1 Структуры данных

`index_s` – запись вторичного индекса базы данных. Содержит два поля:

- `time_mark` (тип `double`) – временная метка в модифицированных юлианских днях;
- `recno` (тип `uint64_t`) – номер записи в первичном индексе (первичный ключ). Размер структуры составляет 16 байт, выравнивание естественное.

`index_hdr_s` — заголовок индексного файла. Хранит единственное поле `records` (тип `uint64_t`) – общее количество записей в файле. В файле после заголовка располагается выравнивающий промежуток (`padding`), дополняющий длину заголовка до границы страницы, после чего следует непрерывный массив структур `index_s`.

`shared_data_t` — общий контекст, разделяемый всеми рабочими потоками и главным потоком. Включает:

- `buffer` – указатель на область памяти, отображённую из файла (массив записей);
- `total_records` – общее количество записей в текущем обрабатываемом куске;

- `block_size` – размер одного блока в байтах (кратен размеру `index_s`);
 - `blocks` – количество блоков разбиения буфера;
 - `block_map` – динамический массив состояний блоков (0 – свободен, 1 – занят, 2 – отсортирован);
 - `map_mutex` – мьютекс для защиты `block_map` при конкурентном получении свободных блоков;
 - `start_barrier` – барьер на `threads+1` участников для одновременного старта обработки куска;
 - `merge_barrier` – барьер на `threads` участников для синхронизации рабочих потоков между раундами слияния и при переходе от фазы сортировки к слиянию;
 - `finish_barrier` – барьер на `threads+1` участников для сигнализации главному потоку о завершении обработки куска;
 - `threads` – общее количество рабочих потоков;
 - `temp_buffer` – временный буфер того же размера, что и `buffer`, используемый при слиянии пар блоков;
 - `exit_flag` – флаг, указывающий рабочим потокам на необходимость завершения (принимает значение 0 или 1).
- `thread_arg_t` — аргумент, передаваемый каждому рабочему потоку при создании. Содержит указатель на общий контекст `shared` и уникальный идентификатор потока `id` (целое число от 0 до `threads-1`).

2.2 Функции главного потока

`int main(int argc, char *argv[])` — точка входа программы:

- Разбор аргументов командной строки: ожидает четыре числовых параметра и имя файла. Проверяет корректность значений: `memsize` кратен размеру страницы; `blocks` является степенью двойки и не менее $4 * threads$; `threads` лежит в диапазоне от 1 до $8 * num_cores$ (где `num_cores` — число процессорных ядер в системе).
- Подготовка файла: открывает индексный файл в режиме чтения/записи, считывает заголовок `index_hdr_s`, вычисляет размер данных и проверяет, что он не превышает `memsize`.
- Инициализация контекста синхронизации: выделяет память под карту блоков (`block_map`) и временный буфер, инициализирует мьютекс `map_mutex` и три барьера.
- Запуск рабочих потоков: создаёт `threads` потоков, выполняющих функцию `worker_routine`. Каждому потоку передаётся его порядковый номер (`id`) и указатель на общий контекст.
- Отображение файла в память: с помощью `mmap` отображает область данных файла (со смещения, кратного странице) в адресное пространство процесса.

- Запуск обработки: заполняет поля `shared` (указатель на буфер, количество записей, размер блока и т.п.), обнуляет карту блоков и проходит через `start_barrier`, запуская рабочие потоки.

- Ожидание завершения куска: дожидается на `finish_barrier`, пока все потоки не завершат сортировку и слияние. Затем синхронизирует изменения с диском (`msync`) и снимает отображение (`munmap`).

- Завершение потоков: устанавливает `exit_flag = 1`, ещё раз проходит `start_barrier` (чтобы вывести потоки из ожидания старта следующего куска) и присоединяет все потоки с помощью `pthread_join`.

- Освобождение ресурсов: уничтожает мьютекс, барьеры, освобождает выделенную память и закрывает файловый дескриптор.

Вспомогательные утилиты (в составе проекта):

- `gen` — программа генерации неотсортированного индексного файла. Принимает количество записей и имя файла. Генерирует записи с последовательным `resno` и случайной `time_mark` в диапазоне от 15020.0 (1900-01-01) до «вчерашней» модифицированной юлианской даты. Формирует заголовок, выравнивание и массив записей.

- `view` — программа просмотра содержимого индексного файла. Выводит количество записей, а затем построчно значения `time_mark` и `resno` для каждой записи.

2.3 Функции рабочих потоков

`static void* worker_routine(void* arg)` — основная функция потока-сортировщика. Параметр: указатель на `thread_arg_t`, из которого извлекаются `shared` и `id`. Работает в бесконечном цикле, на каждой итерации обрабатывая один кусок данных:

Шаг 1. Ожидает на `start_barrier`. Если `exit_flag` установлен, немедленно выходит из цикла и завершает поток.

Шаг 2. Фаза сортировки блоков:

- Если `id < blocks`, поток сразу сортирует «свой» блок с номером `id` при помощи библиотечной функции `qsort` и помечает его в `block_map` как отсортированный (код 2).

- Затем входит в цикл конкурентного захвата свободных блоков: под защитой `map_mutex` ищет блок с состоянием 0, помечает его как 1 (занят), освобождает мьютекс и сортирует этот блок. После сортировки повторно захватывает мьютекс и меняет состояние блока на 2. Если свободных блоков не осталось, выходит из цикла.

- Ожидает на `merge_barrier`, синхронизируясь с другими потоками.

Шаг 3. Фаза слияния блоков:

- Выполняется итеративно: на каждом шаге количество блоков `cur_blocks` сокращается вдвое, а размер каждого блока (в записях) удваивается.

- Потоки с `id < cur_blocks/2` участвуют в слиянии: поток с номером `id` сливает два блока — левый ($2*id$) и правый ($2*id+1$) — во временный буфер `temp_buffer`, используя функцию `merge_blocks`.

- После первого раунда слияния все потоки ожидают на `merge_barrier`. Затем только поток 0 копирует всё содержимое `temp_buffer` обратно в основной `buffer` (чтобы подготовить данные к следующему раунду). После этого — ещё одно ожидание на `merge_barrier`, чтобы гарантировать завершение копирования перед следующим раундом.

- Значения `cur_blocks` и `cur_sz` обновляются, и если `cur_blocks > 1`, раунды повторяются.

Шаг 4. Когда `cur_blocks` становится равным 1 (все данные слиты в один отсортированный блок), поток проходит `finish_barrier`, сигнализируя главному потоку о завершении куска, и возвращается к ожиданию следующего `start_barrier`.

`static int compare_index(const void* a, const void* b)` — компаратор для `qsort` и операций слияния. Сравнивает две записи типа `index_s` по возрастанию `time_mark`, а при равенстве временных меток — по возрастанию `resno`.

`static void merge_blocks(index_s* dest, index_s* src1, size_t n1, index_s* src2, size_t n2)` — функция слияния двух упорядоченных массивов записей в один упорядоченный массив. Использует стандартный алгоритм двух указателей и компаратор `compare_index`.

3 ПОРЯДОК СБОРКИ И ИСПОЛЬЗОВАНИЯ

Комманда для сборки проекта: `make`.

Комманда для сборки проекта без `Makefile`:

```
gcc -std=c11 -pedantic -Wall -Wextra -Wno-unused-parameter
-Wno-unused-variable -pthread -o sort_index sort_index.c
gcc -std=c11 -pedantic -Wall -Wextra -Wno-unused-parameter
-Wno-unused-variable -o gen gen.c -lm
gcc -std=c11 -pedantic -Wall -Wextra -Wno-unused-parameter
-Wno-unused-variable -o view view.c
```

Использование:

```
./gen <количество_записей> <имя_файла>
```

```
./view <имя_файла>
```

```
./sort_index <memsize> <blocks> <threads> <имя_файла>
```

4 МЕТОД И РЕЗУЛЬТАТ ТЕСТИРОВАНИЯ

Комманды для тестирования:

make valtest

Результат выполнения команд для тестирования:

```
=== Valgrind memory check ===
./gen 512 test.idx
Generated file 'test.idx' with 512 records
--- Running sort_index under Valgrind ---
valgrind --leak-check=full --show-leak-kinds=all --error-
exitcode=1 \
    ./sort_index 16384 32 8 test.idx
==31902== Memcheck, a memory error detector
==31902== Copyright (C) 2002-2024, and GNU GPL'd, by Julian
Seward et al.
==31902== Using Valgrind-3.26.0 and LibVEX; rerun with -h for
copyright info
==31902== Command: ./sort_index 16384 32 8 test.idx
==31902==
==31902==
==31902== HEAP SUMMARY:
==31902==       in use at exit: 0 bytes in 0 blocks
==31902==   total heap usage: 12 allocs, 12 frees, 18,880 bytes
allocated
==31902==
==31902== All heap blocks were freed -- no leaks are possible
==31902==
==31902== For lists of detected and suppressed errors, rerun
with: -s
==31902== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
--- Checking sort order ---
SUCCESS: sorted
```